LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 16

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

Bab 1. Pendahuluan

1.1 Latar Belakang

Pada praktikum sebelumnya, kita sudah membuat API untuk data mahasiswa pada aplikasi Sistem Informasi Akademik (si-akademik). API tersebut baru bisa mengambil data (GET) dan menambah data (POST**).**

Pada praktikum ini, API tersebut dikembangkan menjadi RESTful API. RESTful API adalah API yang memakai resource (data, misalnya mahasiswa), endpoint (alamat), dan HTTP method (GET, POST, PUT, DELETE) untuk mengelola data lewat HTTP. Dengan begitu, data mahasiswa bisa dilihat, ditambah, diubah, dan dihapus. Agar API bisa dicoba tanpa membuat tampilan (frontend) terlebih dahulu, kita memakai Postman untuk mengirim request dan melihat response dari API.

1.2 Tujuan Praktikum

1. Memahami RESTful API, endpoint, request, dan response.
2. Membuat endpoint GET, GET berdasarkan ID, POST, PUT, dan DELETE untuk data mahasiswa.
3. Menggunakan JSON sebagai format request dan response.
4. Menggunakan HTTP status code yang sesuai.
5. Menguji semua endpoint dengan Postman, termasuk kondisi error.

Bab 2. Dasar Teori

2.1 RESTful API

RESTful API adalah API yang menggunakan prinsip REST (Representational State Transfer) untuk menyediakan dan mengelola data melalui HTTP. Data biasanya disebut resource. Pada Sistem Informasi Akademik, contoh resource adalah mahasiswa, prodi, dosen, dan mata_kuliah. Setiap resource punya endpoint, misalnya /api/mahasiswa, /api/prodi, dan /api/dosen.

Contoh alur CRUD pada REST API:

Request	Artinya
GET /api/mahasiswa	Mengambil semua mahasiswa
GET /api/mahasiswa/1	Mengambil mahasiswa dengan ID 1
POST/api/mahasiswa	Menambah mahasiswa
PUT /api/mahasiswa/1	Mengubah mahasiswa dengan ID 1
DELETE/api/mahasiswa/1	Menghapus mahasiswa dengan ID 1

2.2 Endpoint

Endpoint adalah alamat yang dipakai untuk mengakses fungsi tertentu pada API. Contoh: http://localhost/si-akademik/api/mahasiswa

Endpoint bisa memakai parameter. Pada /api/mahasiswa/1, angka 1 adalah ID mahasiswa yang ingin diakses.

2.3 Request

Request adalah permintaan yang dikirim client (misalnya Postman) kepada API. Request terdiri dari:

- HTTP Method
- URL atau endpoint
- Header
- Parameter
- Body

2.4 Response

Response adalah jawaban yang diberikan API kepada client. Contoh response berhasil:


```
{
   "success": true,
   "message": "Data berhasil diambil",
   "data": {
      "id": 4,
      "nim": "251004",
      "nama": "Nila Agustin Prima Aurelia",
      "email": "nilaaurelia97@gmail.com",
      "prodi_id": 2,
      "angkatan": "2025",
      "status": "aktif"
   }
```


Status	Arti
200	Request berhasil
201	Data berhasil dibuat
400	Request tidak valid

404	Data atau endpoint tidak ditemukan
500	Terjadi error pada server
405	Method tidak didukung
409	NIM sudah terdaftar

2.5 Postman

Postman adalah aplikasi untuk membuat, mengirim, dan menguji HTTP request ke API. Dengan Postman, kita bisa menguji API tanpa membuat frontend lebih dulu.

Bab 3. Hasil Praktikum

3.1 Studi Kasus

API Data Mahasiswa dari praktikum sebelumnya dikembangkan menjadi RESTful API dengan 5 endpoint berikut:

Pada praktikum ini, alamat yang dipakai di komputer adalah:

BASE_URL = http://localhost/Web_Server/Acara%2016/si-akademik/api/mahasiswa.php

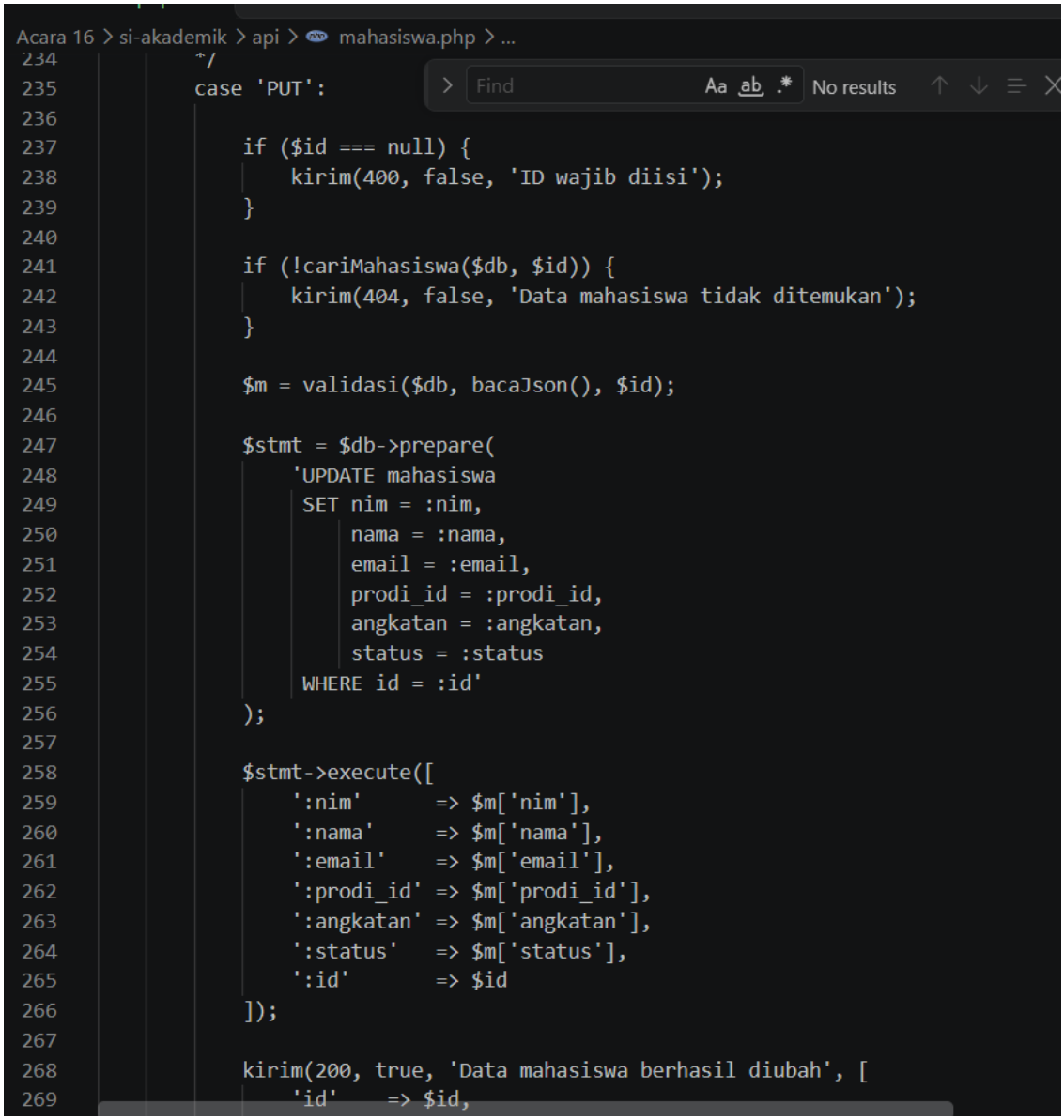
Jadi, contohnya GET berdasarkan ID ditulis BASE_URL/1. Alamat memakai .php karena selama praktikum tidak ada file baru yang dibuat.

3.2 Alat dan Bahan

- Laragon (Apache dan MySQL)
- Folder project: D:\laragon\www\Web_Server\Acara 16\si-akademik
- phpMyAdmin dan database si_akademik (tabel mahasiswa dan prodi)
- Postman
- Code editor [VS Code]

3.3 Mengubah file api/mahasiswa.php

a. PUT


Acara 16 > si-akademik > api > mahasiswa.php > ...
Find
Aa ab .*
No results
*/
case 'PUT':
if ($id === null) {
kirim(400, false, 'ID wajib diisi');
}
if (!cariMahasiswa($db, $id)) {
kirim(404, false, 'Data mahasiswa tidak ditemukan');
}
$m = validasi($db, bacaJson(), $id);
$stmt = $db->prepare(
'UPDATE mahasiswa
SET nim = :nim,
nama = :nama,
email = :email,
prodi_id = :prodi_id,
angkatan = :angkatan,
status = :status
WHERE id = :id'
);
$stmt->execute([
':nim' => $m['nim'],
':nama' => $m['nama'],
':email' => $m['email'],
':prodi_id' => $m['prodi_id'],
':angkatan' => $m['angkatan'],
':status' => $m['status'],
':id' => $id
]);
kirim(200, true, 'Data mahasiswa berhasil diubah', [
'id' => $id,

b. DELETE

```
| DELETE -> hapus data berdasarkan ID
|--------------------------------------------------------------
*/
case 'DELETE':

    if ($id === null) {
        kirim(400, false, 'ID wajib diisi');
    }

    if (!cariMahasiswa($db, $id)) {
        kirim(404, false, 'Data mahasiswa tidak ditemukan');
    }

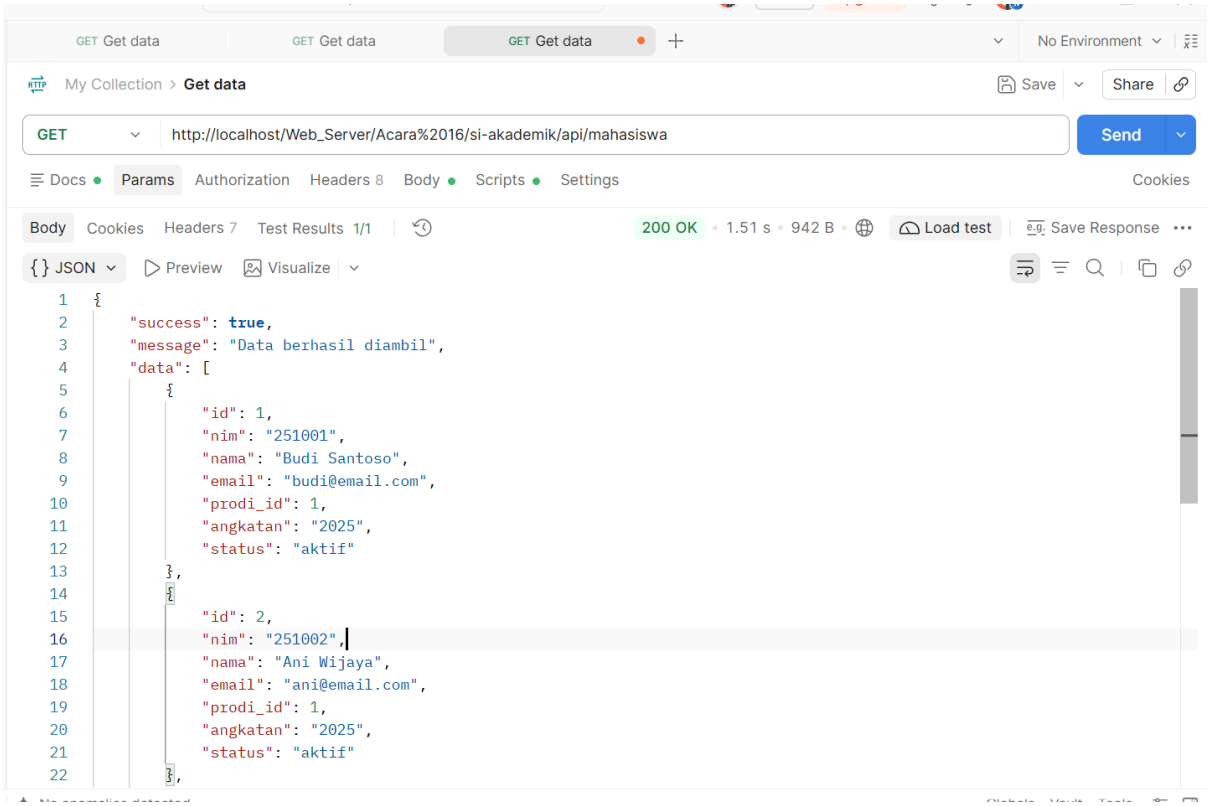
    $stmt = $db->prepare('DELETE FROM mahasiswa WHERE id = :id');
    $stmt->execute([':id' => $id]);

    kirim(200, true, 'Data mahasiswa berhasil dihapus');
    break;
```

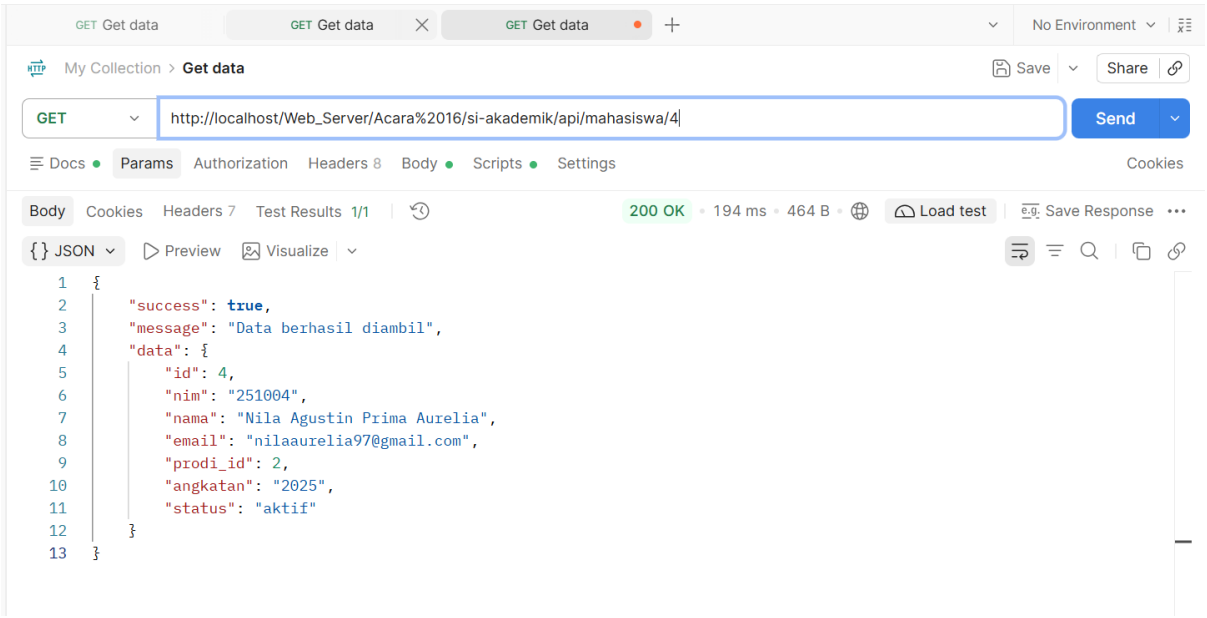
3.4 Hasil Pengujian

a. Pengujian Berhasil

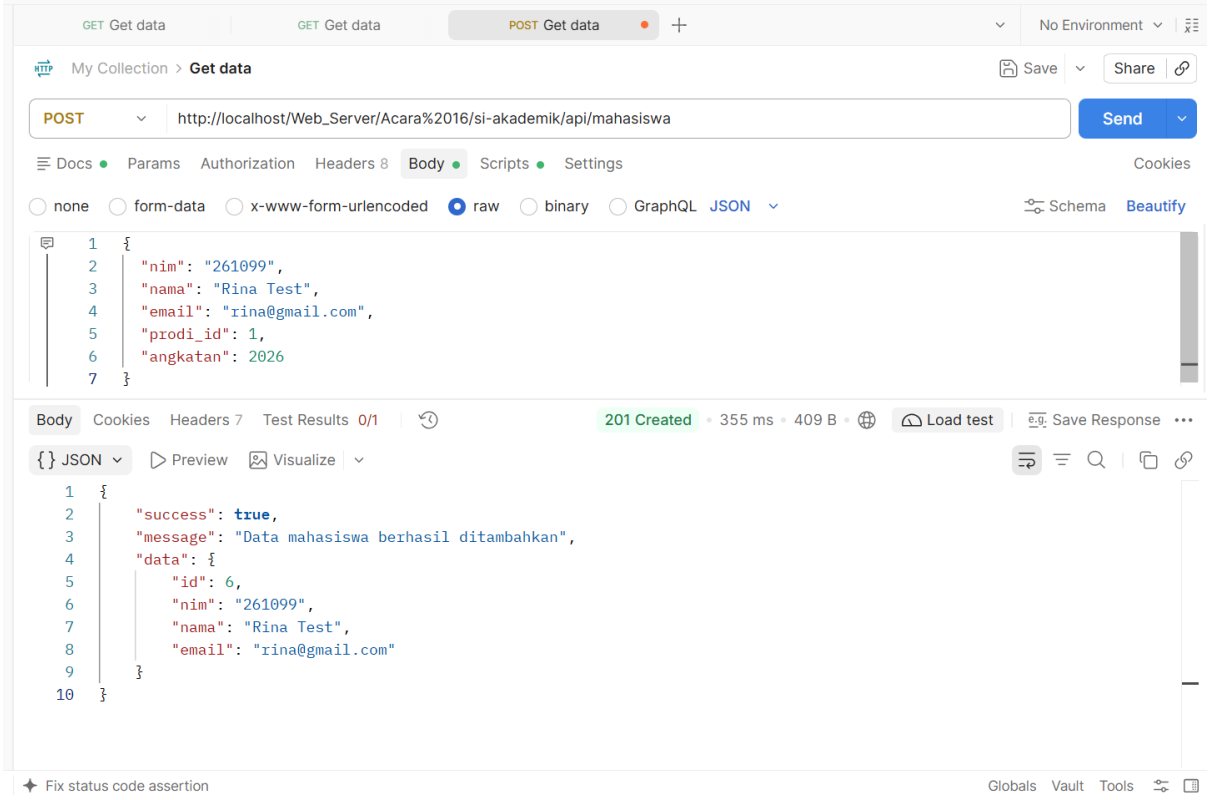
- Menampilkan semua mahasiswa



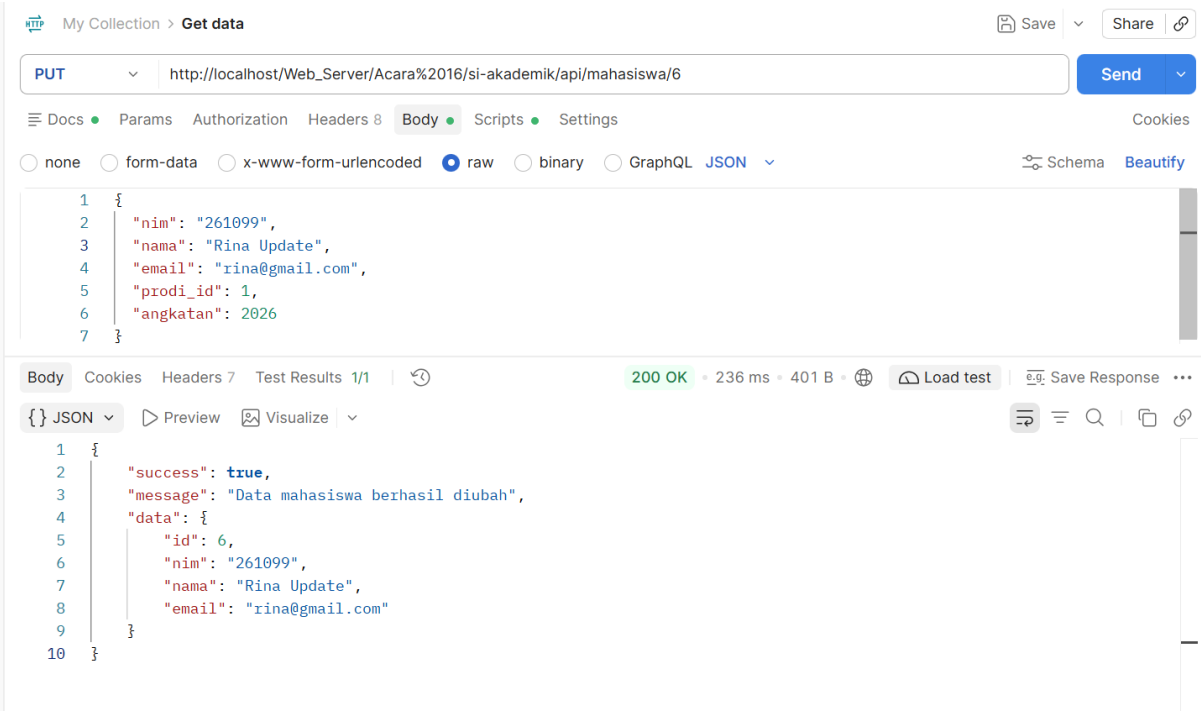
- Menampilkan mahasiswa berdasarkan ID



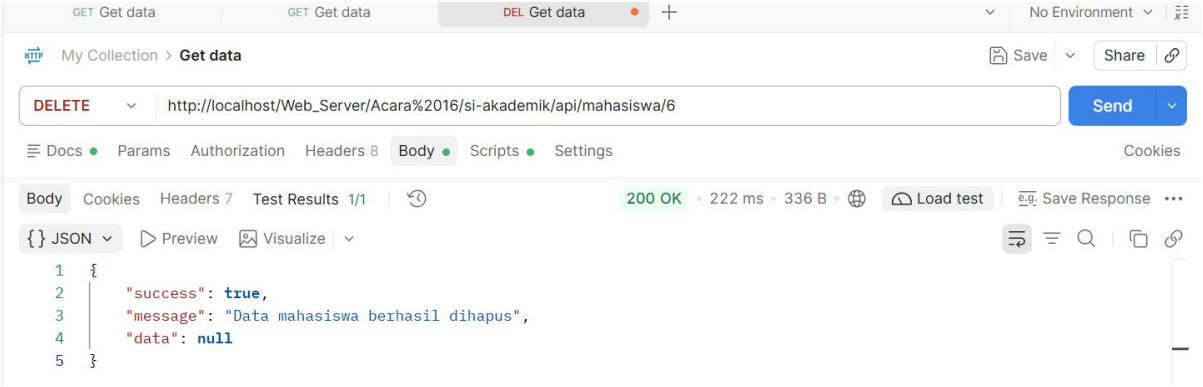
- Menambah mahasiswa



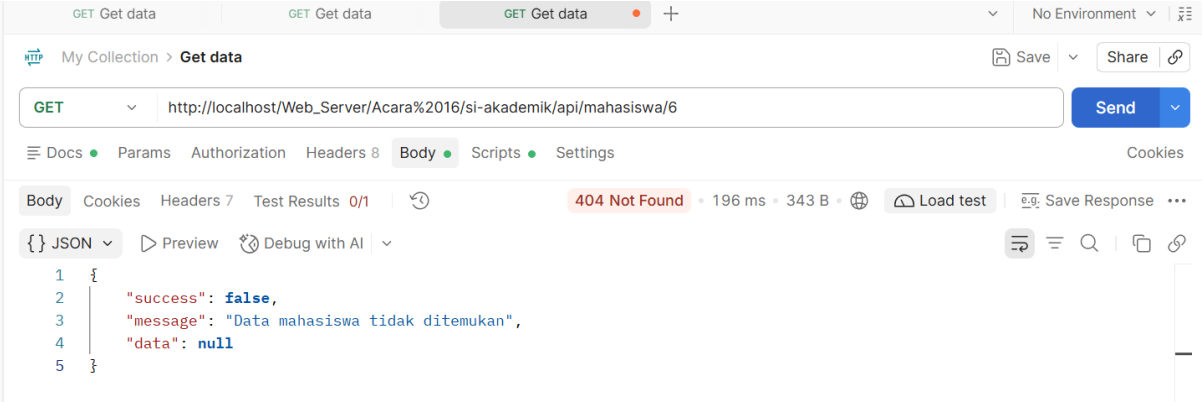
- Mengubah mahasiswa baru (ID dari hasil post)



- Menghapus mahasiswa baru

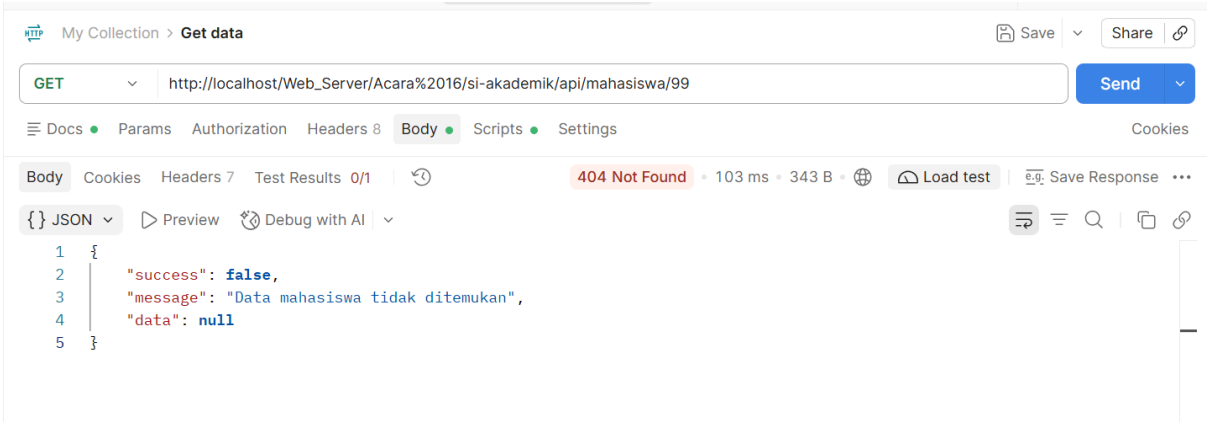


- Mengecek data yang sudah dihapus

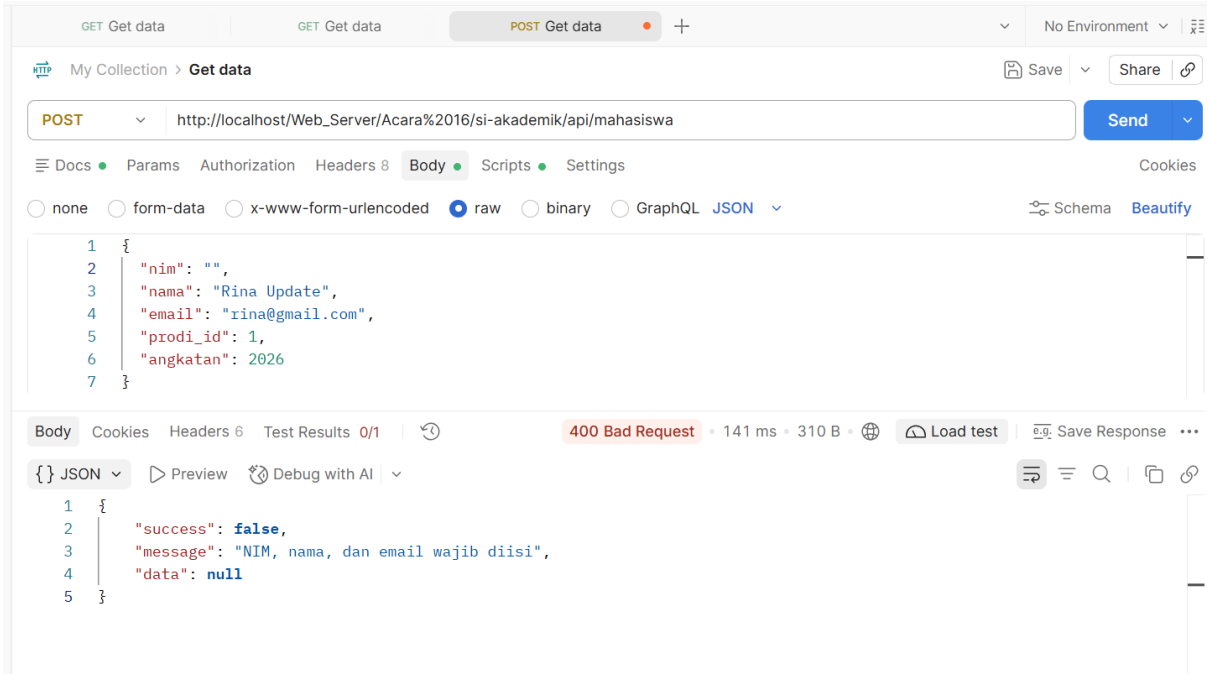


b. Pengujian kondisi error

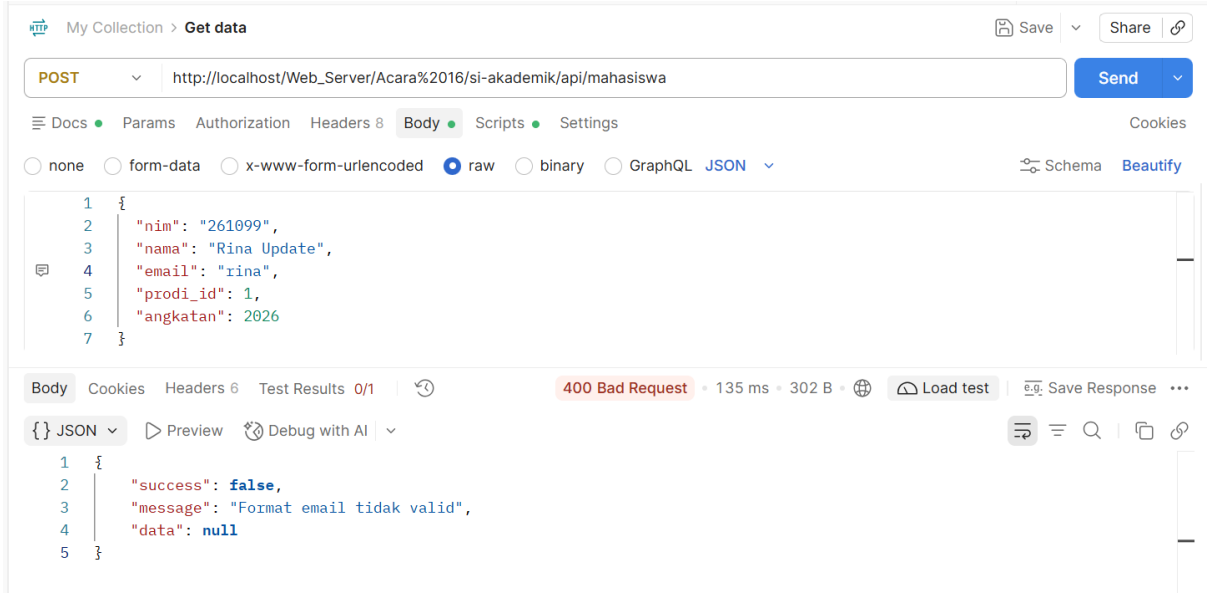
- ID tidak ditemukan



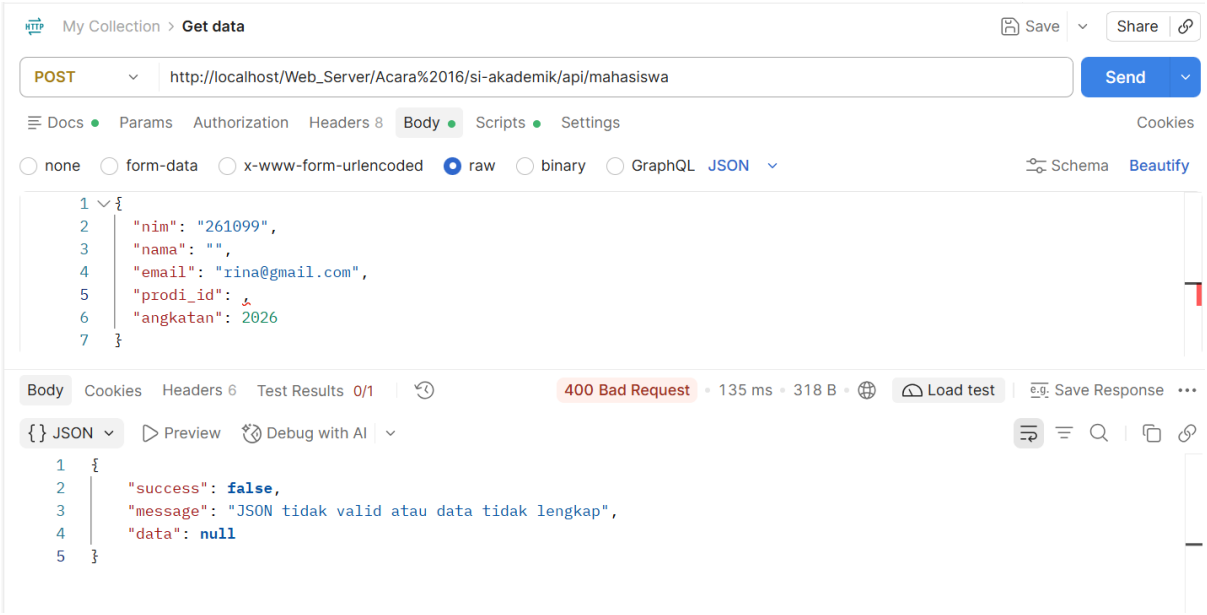
- NIM kosong



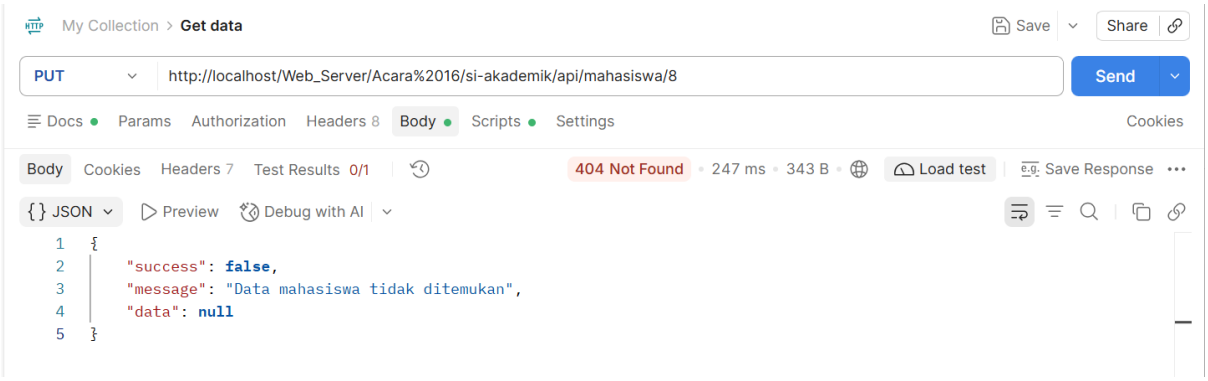
- Email tidak valid



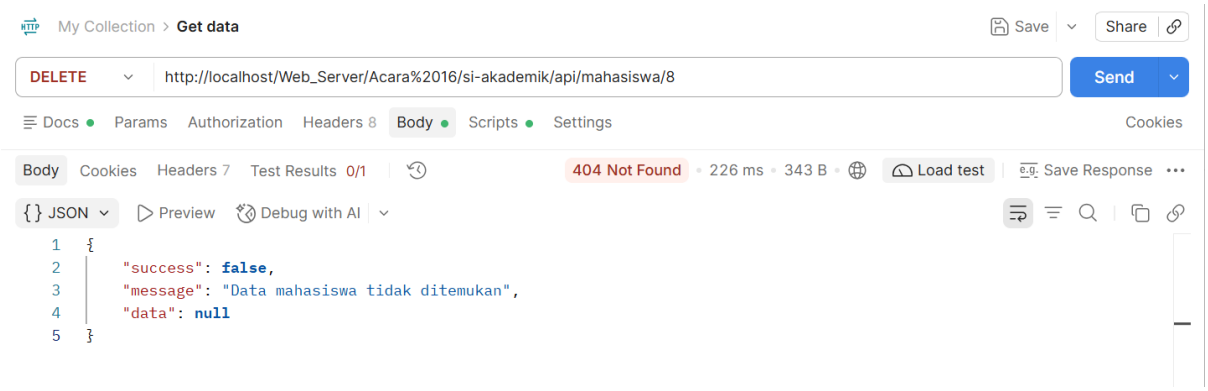
- Data JSON tidak lengkap



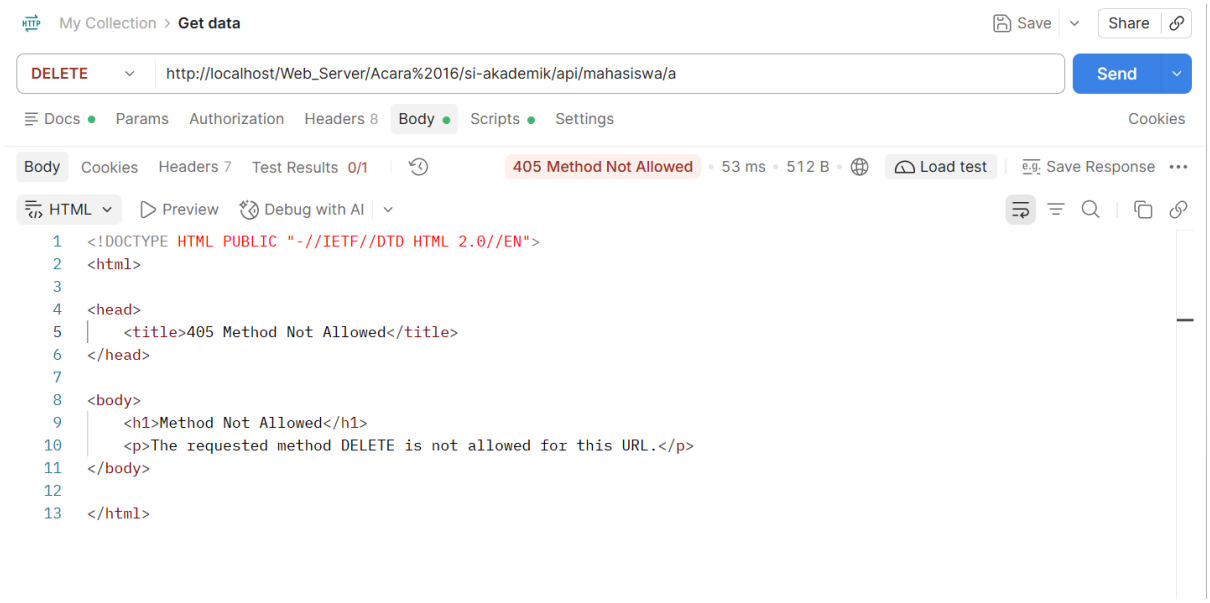
- ID tidak ditemukan saat PUT



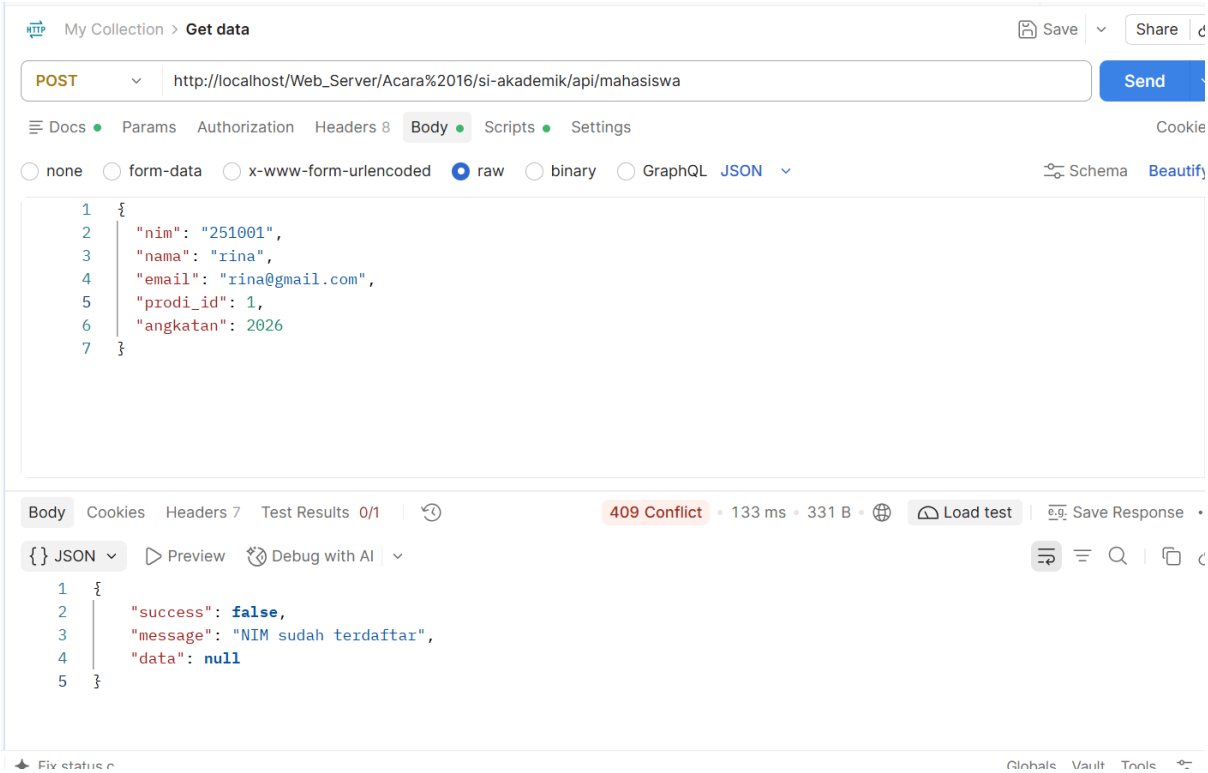
- ID tidak ditemukan saat DELETE



- ID bukan angka



- NIM sudah dipakai



Bab 4. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa RESTful API bekerja dengan menggabungkan endpoint, HTTP method, dan request. Client, dalam hal ini Postman, mengirim request ke endpoint tertentu. API kemudian memprosesnya, mengambil atau mengubah data di database, lalu mengirim kembali response dalam bentuk JSON. Alamat yang sama, misalnya /api/mahasiswa, bisa dipakai untuk tindakan yang berbeda karena yang

menentukan adalah HTTP method-nya: GET untuk mengambil data, POST untuk menambah, PUT untuk mengubah, dan DELETE untuk menghapus.

API data mahasiswa pada si-akademik berhasil dikembangkan menjadi RESTful API dengan lima endpoint, yaitu GET semua mahasiswa, GET berdasarkan ID, POST, PUT, dan DELETE. Pengembangan ini hanya dilakukan dengan mengubah satu file, yaitu api/mahasiswa.php, tanpa membuat file baru. API memakai JSON untuk request dan response, serta HTTP status code yang sesuai, misalnya 200 untuk berhasil, 201 untuk data berhasil dibuat, 400 untuk request yang tidak valid, dan 404 untuk data yang tidak ditemukan.

Hasil pengujian dengan Postman menunjukkan bahwa semua endpoint berjalan sesuai rancangan. Pada kondisi error, seperti ID tidak ditemukan, NIM kosong, email tidak valid, dan data JSON tidak lengkap, API tidak berhenti dengan error server, melainkan menjawab dengan status code dan pesan yang jelas. Dengan demikian, Postman terbukti berguna untuk memeriksa apakah API bekerja sesuai dengan request dan response yang dirancang, tanpa harus membuat tampilan frontend terlebih dahulu.